

Distributed Systems for MMORPG Backends

Consistency · Ownership · Fault Tolerance · Recovery

A self-study guide · ~30 min read

Everything you need to reason clearly about state, ownership, and failure in a distributed game server architecture.

Contents

- 1. The Landscape of Distributed Systems**
Core vocabulary and the CAP theorem
- 2. Database Architecture — One DB vs. Many**
Why a single logical DB wins
- 3. Ownership and Leases**
Who is allowed to write, and for how long
- 4. Fencing Tokens**
Preventing zombie writers
- 5. The Handoff Protocol**
Transferring ownership when a player changes zone
- 6. Recovery and Reconciliation**
Getting back to truth after failure
- 7. Patterns and Toolbox**
Event sourcing, sagas, CRDTs, and more
- 8. Quick-Reference Cheat Sheet**
Everything on one page

The Landscape of Distributed Systems

Core vocabulary and the CAP theorem

What is a distributed system?

A distributed system is a collection of independent computers that appear to users as a single coherent system. In your MMORPG, this means your auth server, battle server, multiple world servers, and databases — all cooperating to give each player a seamless experience. The challenge is that each node can fail independently, the network between them is unreliable, and there is no shared clock or shared memory.

Key vocabulary

Node — Any individual server or process in the system.

Partition — A network failure that prevents some nodes from communicating.

Consistency — Every read reflects the most recent write.

Availability — Every request receives a response (not necessarily the latest data).

Fault tolerance — The system continues operating despite node or network failures.

Linearizability — The strongest form of consistency — operations appear instantaneous and ordered.

Eventual consistency — Nodes may diverge temporarily but will converge given enough time.

Idempotency — Applying the same operation multiple times produces the same result as applying it once.

The CAP Theorem

Eric Brewer's CAP theorem states that a distributed system can guarantee at most two of the following three properties simultaneously:

- **Consistency** — all nodes see the same data at the same time.
- **Availability** — every request gets a response.
- **Partition tolerance** — the system works even if messages are lost.

Practical implication for your game: Network partitions are unavoidable in production — you must tolerate them. This means you are always choosing between Consistency and Availability when a partition occurs. For game currency and inventory, prefer consistency (don't accept writes you can't guarantee). For position or chat, prefer availability (a slightly stale position is fine).

The consistency spectrum

Consistency is not binary. From strongest to weakest:

Level	Meaning
Linearizability	Every operation appears to execute atomically at a single point in time. Safe but slow — requires coord
Sequential consistency	All nodes see operations in the same order, though not necessarily real-time. Slightly weaker but more

Causal consistency	Operations that are causally related appear in order. Unrelated ops may differ. Good middle ground.
Eventual consistency	Nodes will agree — eventually. No guarantee on when. Best throughput, highest risk for game state.

Database Architecture

One DB vs. per-server databases

One of the first architectural decisions in a distributed game backend is whether each world server manages its own isolated database, or whether all servers share a single logical database.

The per-server DB approach (and why it fails)

At first glance, giving each world server its own DB seems clean: no contention, no single point of failure. In practice:

- Player-to-player interactions across zone boundaries require cross-shard queries — complex and slow.
- When a player moves zones, their data must be physically migrated between DBs, introducing a failure window.
- Global queries (leaderboards, guild membership, auction houses) become nightmares.
- Schema migrations must be applied to every shard independently.

The single logical DB approach

A single DB (or a logically unified distributed DB like CockroachDB or Spanner) avoids all of the above. The critical insight is:

The database is not your consistency boundary — your servers are. The DB is just durable storage. Correctness is enforced by controlling who is allowed to write to each row, not by isolating the storage itself.

World servers become stateless processors that read from and write to the shared DB, but only for players they are authoritative over. This is enforced through the ownership and lease system described in the next chapter.

What goes in the DB

Data	Storage	Reason
Player state (HP, position, inventory)	DB	Durable, needs to survive crashes
Ownership / lease metadata	DB	Must be visible to all servers
Event log / write-ahead log	DB	Needed for recovery
Zone entity list (NPCs, objects)	DB or world server memory	Can be reloaded on restart
Session routing (player → server)	Coordination service or DB	Needs fast reads
Ephemeral combat state	World server memory only	Acceptable to lose on crash

Ownership and Leases

Who is allowed to write, and for how long

With a shared DB, multiple servers could theoretically write to the same player row simultaneously. We need a mechanism to designate a single authoritative writer at any given time. This is ownership.

Naive ownership — just an owner field

A simple approach is to store the owning server's ID on each player row:

```
players: { player_id, owner_server_id, hp, position, inventory, ... }
```

A server only writes to rows where `owner_server_id` matches its own ID. The problem: if the owning server crashes, nobody else can ever take over. The owner field is permanent until manually changed.

Leases — time-bounded ownership

A lease adds an expiry to ownership. A server is authoritative only if it is the owner AND the lease has not expired:

```
players: { player_id, owner_server_id, lease_expires_at, hp, position, ... }
```

The rules:

- A server may write only if: `owner_server_id = self AND lease_expires_at > now()`
- The server must renew its lease periodically before it expires.
- If a server crashes, its lease expires naturally — other servers can then claim it.
- Lease duration is a tradeoff: shorter = faster failover, more renewal overhead. Typically 5–30 seconds for game backends.

Writing with lease enforcement

Every write must be conditional — never a blind UPDATE:

```
UPDATE players SET hp = $new_hp, lease_expires_at = now() + interval '10 seconds' WHERE
player_id = $id AND owner_server_id = $my_server_id AND lease_expires_at > now();
```

If 0 rows are affected, the server has lost its lease and must stop processing that player immediately. This is the critical safety check.

Never ignore the row count on writes. Checking `rows_affected = 1` is what separates a correct distributed system from one that silently corrupts data. If your write returns 0, treat it as a fatal error for that player's session.

Claiming an expired lease

When a server wants to take ownership of a player whose lease has expired (either because the owner crashed or the player is reconnecting), it uses a compare-and-swap operation:

```
UPDATE players SET owner_server_id = $my_id, lease_expires_at = now() + interval '10
seconds' WHERE player_id = $id AND (owner_server_id IS NULL OR lease_expires_at < now());
```

Only one server will win this race — the DB serializes concurrent updates. The server that gets `rows_affected = 1` is the new owner.

Fencing Tokens

Preventing zombie writers definitively

Leases handle the normal case, but there is a subtle failure mode they don't fully address: the zombie server.

The zombie server problem

Imagine Server A owns a player. A gets hit by a GC pause or network hiccup and stops renewing its lease. The lease expires. Server B claims ownership and starts writing. Then A wakes up — it still thinks it's the owner, and it has pending writes queued up. If A doesn't check its lease on every write (or if there's a clock skew), it could write stale or conflicting data right on top of B's valid updates.

Leases rely on clocks. Clocks drift. A server that is paused (GC, VM migration, kernel scheduling) can wake up and be unaware that time has passed and its lease expired. Never assume your wall clock is reliable as a sole safety mechanism.

Fencing tokens — generation-based authority

A fencing token is a monotonically increasing integer issued every time ownership changes hands. The DB rejects any write whose token is lower than the current token stored — regardless of what the writing server thinks:

```
players: { player_id, owner_server_id, fence_token, lease_expires_at, ... }
```

How it works:

1. Each time ownership is granted, `fence_token` is incremented atomically (`fence_token = fence_token + 1`).
2. The new owner receives the current `fence_token` value.
3. Every write includes that token in the WHERE clause:

```
UPDATE players SET hp = $hp WHERE player_id = $id AND fence_token = $my_token AND
owner_server_id = $my_id AND lease_expires_at > now();
```

4. The zombie server still has the old token (e.g. 42). The new owner has token 43. The zombie's writes are rejected.

Leases vs. fencing tokens — do you need both?

	Leases	Fencing Tokens
Protects against	Permanent stuck ownership after crash	Zombie writers with stale belief
Relies on	Clocks (can drift)	Monotonic counter (always correct)
Failover speed	Lease TTL duration	Immediate (token already incremented)
Recommended	Yes	Yes — together they cover all cases

The Handoff Protocol

Transferring ownership when a player changes zone

Crash-induced ownership transfer is one thing — you just wait for the lease to expire. A planned handoff between two live servers is more delicate, because you want to minimize the gap (time when nobody writes) while eliminating any overlap (both servers writing simultaneously).

The two failure modes to eliminate

- **Gap:** A moment where neither server will accept writes for the player. The client sends an action; both servers reject it.
- **Overlap:** Both servers accept writes simultaneously, leading to diverged state.

The four-phase handoff protocol

1 — PREPARE

Server A (outgoing) marks the player's status as 'transferring' in the DB and flushes all pending in-memory state. While in this status, A stops accepting new actions from the client. This is the write freeze.

2 — GRANT

Server B (incoming) issues a compare-and-swap to claim ownership:

```
SET owner=B, fence_token=N+1 WHERE owner=A AND status='transferring'
```

If this succeeds, B is now the authoritative owner with a new fence token.

3 — CONFIRM

B loads the player's full state from the DB and begins accepting client connections and write requests. The player's client is instructed to reconnect to Server B's address.

4 — RELEASE

A receives confirmation that B is live, clears the player from its local state, and the transfer is complete. If A never receives this confirmation, it can poll the DB — once it sees owner=B, it knows the transfer succeeded.

Edge cases

Server A crashes during PREPARE — the player is stuck in 'transferring' with no owner. This is why the lease is still active during transfer. When the lease expires, the player is claimable by any server or watchdog process.

Client disconnects mid-handoff — this is actually the easiest case. Complete the transfer anyway. When the client reconnects, it queries the routing service for which server owns it and connects to whoever that is now.

A and B can't reach each other — this doesn't matter. They never communicate directly. The DB is the sole coordination point. As long as both can reach the DB, the protocol works.

The border zone optimization

In most MMORPGs, zone boundaries have overlap regions. When a player enters the border area of Zone A approaching Zone B, Server B begins receiving the player's data in read-only mode — it pre-loads their state, caches their inventory, pre-warms whatever it needs. When the actual ownership transfer fires, Server B is already ready. From the player's perspective the handoff is nearly instantaneous. This is a latency optimization layered on top of the correctness protocol — correctness comes first.

Two-Phase Commit (2PC): The handoff protocol above is structurally similar to 2PC — a classical algorithm for atomic operations across distributed nodes. In pure 2PC there is a coordinator that can become a bottleneck or single point of failure. The DB-as-coordinator approach here avoids that by using the DB's own atomic operations as the coordination mechanism.

Recovery and Reconciliation

Getting back to truth after failure

Recovery is what happens when a server realizes it is in an inconsistent state and needs to return to a known-good truth. Reconciliation is the process of resolving conflicting state between two parties. They usually happen together.

Establishing your source of truth

Before you can recover, you need to define what 'truth' is. Your options:

DB is truth — the persisted DB state is canonical. Server memory is just a working cache. On crash/restart, servers reload from DB. Anything not yet flushed is lost. Simple and robust — the recommended default.

Event log is truth — you store every state-changing event (player moved, took damage, picked up item) rather than the current state. State is always reconstructed by replaying the log. This is Event Sourcing, covered in Chapter 7. More complex, but gives you complete history and clean reconciliation.

The less stateful your servers are — the more they treat the DB as truth and themselves as processors — the easier recovery becomes. Every megabyte of in-memory state that hasn't been flushed is technical debt you'll pay at 3am when a server crashes.

Case 1 — The zombie server realizes it lost its lease

A server's write returns 0 rows. It must:

1. **STOP** — immediately halt all writes for affected players. No more processing.
2. **DIFF** — compare in-memory state against the DB. What was accepted? What was rejected?
3. **DECIDE** — apply field-level resolution rules (see below).
4. **DISCARD** — drop local state, reload from DB, or hand off to the player's new server.

Field-level resolution rules

Field	Strategy	Reason
Position	DB wins	New owner has moved the player; stale position is wrong
HP (damage taken)	Take the lower value	Conservative — prevents exploits where damage is 'lost'
XP / currency gained	Requires event log	Can't safely merge without knowing what was already credited
Inventory add	Idempotent if using event log	Replay the add event; DB constraint prevents duplicates
Quest progress	DB wins	New server may have advanced it further

Case 2 — Interrupted handoff

A player is stuck in 'transferring' state and the outgoing server is gone. A watchdog process (or the incoming server) must reconstruct which phase failed:

- Was PREPARE complete? Check whether state was fully flushed (no dirty fields).

- Was GRANT issued? Check the fence token — was a new one issued?
- Based on the above, either roll forward (complete the transfer) or roll back (return player to previous server or mark ownerless).

This is why every step of a multi-phase operation should be recorded in a transaction log — so any participant can reconstruct where things were left.

Case 3 — Network partition heals

Two servers were operating in isolation and now can communicate. This is the hardest case, because you may have genuine conflicts.

If your lease and fencing token system is correctly implemented, this case should not produce conflicting writes — only one server should have had a valid lease at any moment. If you find yourself here with real conflicts, treat it as a bug in your ownership enforcement, not a reconciliation problem.

If you do encounter it (perhaps in an early prototype without fencing), your options in order of preference:

- **Per-field merge rules** — domain logic decides which value wins per field (same table as above).
- **Last-write-wins with logical clocks** — use Lamport timestamps or vector clocks rather than wall time to avoid clock-drift errors.
- **CRDTs** — data structures that always merge without conflict, mathematically. Applicable to counters, sets, and registers. See Chapter 7.

Patterns and Toolbox

Event sourcing, sagas, CRDTs, and more

Event Sourcing

Instead of storing the current state of an entity, you store the full sequence of events that led to it. The current state is always derived by replaying the log.

```
events: { event_id, player_id, type, payload, timestamp, fence_token }
```

Benefits for games:

- Complete audit trail — you can replay any player's history.
- Clean reconciliation — replay from last known-good checkpoint.
- Debugging — replay a sequence to reproduce bugs exactly.
- Time travel — roll back to any past state.

The cost is storage and replay time. Mitigate with periodic snapshots: store a full state checkpoint every N events, then only replay from the last snapshot.

CQRS — Command Query Responsibility Segregation

Often paired with event sourcing. The idea: separate the path for writing (Commands) from the path for reading (Queries). Write to an event log; maintain read-optimized projections (materialized views) that are updated asynchronously as events arrive. Leaderboards, guild rosters, and map state are natural CQRS read models.

The Saga Pattern

A saga is a sequence of local transactions where each step publishes an event that triggers the next step. If any step fails, compensating transactions undo the previous steps. This replaces distributed transactions (which require locking across services) with a series of smaller, local ones.

Example — player-to-player trade across zone boundaries:

```
1. Server A: reserve item in player A's inventory → emit ItemReserved
2. Server B: reserve gold in player B's inventory → emit GoldReserved
3. Server A: transfer item to player B → emit ItemTransferred
4. Server B: transfer gold to player A → emit TradeComplete
If step 3 fails: Compensate step 2: release gold reservation
Compensate step 1: release item reservation
```

CRDTs — Conflict-Free Replicated Data Types

CRDTs are data structures designed so that any two replicas can be merged without conflicts, producing a deterministic result. They are mathematically guaranteed to converge.

CRDT Type	Use case in games
G-Counter (grow only)	Total XP ever earned, kill count
PN-Counter (increment/decrement)	Currency balance (with floor at 0)

G-Set (grow only set)	Achievements unlocked, items ever owned
OR-Set (add/remove set)	Current inventory (can add and remove)
LWW-Register (last write wins)	Player display name, settings

Message Queues

Async communication between servers via a message broker (Kafka, RabbitMQ, NATS) decouples services: if the battle server is down, the world server queues events and they're processed when it comes back. Key properties to look for:

- **At-least-once delivery** — messages may be delivered more than once; make handlers idempotent.
- **Ordering guarantees** — Kafka guarantees order within a partition; use `player_id` as partition key to serialize one player's events.
- **Dead letter queues** — messages that fail repeatedly go here for manual inspection rather than silently dropped.

Coordination services

Services like etcd and ZooKeeper implement distributed consensus (via Raft and Zab respectively) and are commonly used for: leader election, distributed locks, service discovery, and configuration. You can implement your lease and ownership system on top of etcd instead of your game DB — etcd's leases are a first-class feature with TTL-based expiry.

Quick-Reference Cheat Sheet

Everything on one page

Core decisions

DB architecture — Single logical DB. Servers control writes, not storage isolation.

Consistency boundary — The owning server. DB is just durable storage.

Ownership mechanism — Lease (time-bounded) + Fencing token (generation-bounded). Both together.

Lease TTL — 5–30 seconds. Shorter = faster failover, more renewal load.

Write enforcement — Every UPDATE must check `owner_server_id`, `fence_token`, AND `lease_expires_at`.

Handoff protocol — PREPARE → GRANT (CAS) → CONFIRM → RELEASE.

Source of truth — DB (simple) or event log (powerful). Don't rely on server memory.

Cross-server operations — Saga pattern — local transactions with compensations.

Things that will burn you if ignored

- Not checking `rows_affected` on every write — silent data corruption.
- Relying on wall clocks alone — clock drift makes leases unreliable.
- Omitting fencing tokens — zombie servers will corrupt data eventually.
- In-memory state that isn't flushed frequently — large recovery gaps.
- Blind UPDATE without lease check — any server can overwrite any player.
- No idempotency on event handlers — duplicate delivery causes double-credit.
- No dead letter queue — failed messages disappear silently.

The schema (minimum viable)

```
CREATE TABLE players ( player_id UUID PRIMARY KEY, owner_server_id VARCHAR, fence_token
BIGINT NOT NULL DEFAULT 0, lease_expires_at TIMESTAMPTZ, transfer_status VARCHAR, -- NULL
| 'transferring' hp INT, position_x FLOAT, position_y FLOAT, -- ... other game fields );
```

Further reading

Designing Data-Intensive Applications — Martin Kleppmann. The definitive book on this entire subject area.

The Chubby Lock Service — Google (Burrows, 2006). Original paper on distributed lock services with leases.

Raft Consensus Algorithm — Ongaro & Ousterhout. Readable consensus algorithm; basis of etcd.

CRDT papers — Shapiro et al.. Conflict-Free Replicated Data Types — formal foundations.

Saga Pattern — Garcia-Molina & Salem. Original paper on sagas as an alternative to distributed transactions.

Good luck with your MMORPG. The hardest distributed systems problems are the ones you don't know you have yet — but now you have the vocabulary to recognize them.